

Centro Universitário de Brasília (CEUB)
Curso de Tecnologia em Análise e Desenvolvimento de Sistemas
Disciplina de Projeto Integrador I

REGIIMENTOWEB – Plataforma Digital & Dashboards

Arquitetura da Solução

Equipe:

José da Conceição Novais Neto
Arthur Amaral dos Santos
Arthur Silas
Matheus Covre Lacerda
Lucas Ferreira de Sousa

2. Arquitetura da Solução – REGIIMENTOWEB

Este documento descreve a arquitetura técnica do protótipo, incluindo camadas do sistema, tecnologias adotadas, estrutura de arquivos e visão de evolução futura.

2.1 Visão Geral da Arquitetura

A solução adota uma arquitetura de três camadas para o MVP, totalmente estática no lado do servidor, o que garante simplicidade, leveza e facilidade de hospedagem gratuita:

■ Camada de Apresentação (Front-end)

HTML5 + CSS3 + Tailwind CSS + JavaScript ES6. Responsável pela interface visual, navegação e experiência do usuário em qualquer dispositivo (Mobile-First).

■ Camada de Dados (JSON Estático)

Arquivos .json locais (publicacoes.json, eventos.json, membros.json, dados.json) que alimentam os componentes dinâmicos via Fetch API. Sem dependência de banco de dados nesta fase.

■ Camada de Visualização (Chart.js)

Biblioteca JavaScript responsável pela renderização dos dashboards interativos — gráficos de barras, linhas e rosca com tooltips e hover interativo.

2.2 Stack Tecnológica

Tecnologia	Versão/Tipo	Finalidade
HTML5	Padrão W3C	Estruturação semântica das páginas
CSS3 + Tailwind CSS	v3.x	Estilização responsiva (Mobile-First)
JavaScript	ES6+	Lógica, manipulação de DOM, Fetch API
Chart.js	v4.x	Gráficos interativos nos dashboards
AOS – Animate On Scroll	v2.x	Animações de entrada de elementos
JSON	Estático	Armazenamento e fornecimento de dados
Git / GitHub	–	Versionamento e colaboração
Vercel / GitHub Pages	Tier Gratuito	Hospedagem e deploy contínuo

2.3 Estrutura de Arquivos

O projeto segue uma organização modular com separação clara de responsabilidades:

```
/index.html
/css/style.css
/js/equipe.js
/js/repositorio.js
/js/eventos.js
/js/dashboards.js
/data/membros.json
/data/publicacoes.json
/data/eventos.json
/data/dados.json
/assets/images/
/README.md
```

2.4 Fluxo de Dados

O fluxo de renderização dinâmica segue o padrão: Evento do usuário (navegação/filtro) → JavaScript executa Fetch API → Arquivo JSON é lido → Dados processados → DOM atualizado → Interface re-renderizada. Não há requisições a servidor externo ou banco de dados, garantindo performance e independência de infraestrutura.

2.5 Arquitetura de Evolução Futura

Componente	Tecnologia Sugerida	Motivo
Back-end / API	Node.js + Express ou Python Flask	Gerenciamento de dados dinâmicos e autenticação
Banco de Dados	MySQL ou PostgreSQL	Persistência de membros, publicações e eventos
Painel Admin	React.js ou Vue.js	Interface de gerenciamento de conteúdo
Autenticação	JWT + OAuth 2.0	Acesso seguro para administradores
CI/CD	GitHub Actions	Deploy automatizado com testes